Low-pass and High-pass Filtering of Sentinel-2 Imagery over the Aravalli Ridge, Gurugram–Faridabad

Which filtering technique suits which remote sensing application?

Group 1

Pranay Chakraborty	MS22041
Sangeeta Kumari	PH25084
Aparna Manoj	MS24070
Robin	MS23138
Sangam	MS23222

A course project report submitted for

EES403: Advanced Remote Sensing and GIS

Instructors: Dr. Chandrakant Ojha and Dr. Meenakshi Devi



Indian Institute of Science Education and Research, Mohali

Abstract

Filtering changes the brightness of each pixel according to the pixels around it. Low-pass filters smooth an image and high-pass filters keep only its edges, and remote sensing textbooks recommend both. We asked which of them is more useful, and for which job. We wrote 43 filter configurations in Python, in the spatial domain (kernel convolution) and in the frequency domain (Fourier transform with masks), and applied them to one Sentinel-2 Level-2A scene of the Aravalli ridge between Gurugram and Faridabad (2024-12-04). Five measurable tests followed: noise removal, land-cover classification against ESA WorldCover, edge and lineament enhancement, change to the spectra, and run time.

No single filter won. A 3×3 median filter removed impulse noise best (36.49 dB against 21.01 dB for the noisy image). A light Gaussian low-pass raised Random Forest classification kappa from 0.403 to 0.502 and halved the speckle in the map. High-pass filters lowered classification accuracy but gave the sharpest images, the best match between detected edges and mapped land-cover boundaries, and, for gradient operators, the clearest ridge lineaments. The ideal frequency mask produced ringing of 9.0% at edges and should be avoided. We therefore recommend light low-pass filtering for quantitative work and high-pass products, kept as separate layers, for interpretation and structural mapping.

Contents

Abstract i

List of Figures iv

List of Tables v

Abbreviations vi

1 Introduction 1

1.1 Spatial frequency 1

1.2 Low-pass and high-pass filters 1

1.3 Aim 2

1.4 How this report is organised 3

2 Study Area and Data 4

2.1 Study area 4

2.2 Satellite data and scene selection 4

2.3 Land-cover reference 5

3 Methods 7

3.1 Workflow 7

3.2 Spatial-domain filters 7

3.3 Frequency-domain filters 9

3.4 Experiments 9

4 Results 12

4.1 What the filters do to the image 12

4.2 Frequency domain and ringing 13

4.3 E1: Noise removal 15

4.4 E2: Land-cover classification 15

4.5 E3 and E4: Edges, lineaments and spectral change 17

4.6 E5: Run time 17

5 Discussion **19**

5.1 Decision matrix 19

5.2 When to use which filter 19

5.3 Recommendation 20

5.4 Limitations 20

6 Summary **22**

A Evaluation Measures in Plain Words **23**

B Where Each Step Lives in the Code **25**

C Reproducing the Results **27**

Bibliography **28**

List of Figures

1.1 Effect of low-pass and high-boost filters on a line profile 2

2.1 Study area and the selected Sentinel-2 scene 5

3.1 Processing flowchart 8

4.1 Low-pass filter results 12
4.2 High-pass filter results 13
4.3 High-boost results 13
4.4 Square ideal masks of 61, 91 and 121 pixels 14
4.5 Ringing of ideal, Butterworth and Gaussian masks 14
4.6 Classification maps 16
4.7 Run time of spatial and frequency filtering 18

List of Tables

2.1 Scene and data summary. 5

3.1 Filter bank: the 43 configurations compared. 10
3.2 The five experiments. 10

4.1 E1: noise removal 15
4.2 E2: land-cover classification 16
4.3 E3 and E4: edges, lineaments and spectral change 18

5.1 Decision matrix 19

A.1 What each evaluation measure tells us. 23

B.1 Map from processing step to code and output. 25

Abbreviations

AOT	Aerosol optical thickness (haze)
DN	Digital number, the raw value stored in a pixel
FFT	Fast Fourier transform
HPF	High-pass filter
LPF	Low-pass filter
NDVI	Normalised difference vegetation index, (NIR $-$ red)/(NIR $+$ red)
NIR	Near-infrared (Sentinel-2 band B8)
PSNR	Peak signal-to-noise ratio
RF	Random Forest classifier
SAM	Spectral angle (between two spectra)
SSIM	Structural similarity index
D_0	Cutoff of a frequency mask, as a fraction of the Nyquist frequency
κ	Cohen's kappa (classification agreement beyond chance)
σ	Width of a Gaussian filter, in pixels

Plain-language definitions of every evaluation measure are in Appendix A.

Chapter 1

Introduction

1.1 Spatial frequency

A satellite image is a grid of numbers. In one part of the image the numbers may stay almost the same from pixel to pixel; in another they may jump up and down every few pixels. *Spatial frequency* is the number of changes in brightness (digital number, DN) per unit distance (Lillesand et al., 2015). A ploughed field or a lake has low spatial frequency: grey values change gradually. A street grid, a ridge crest or a field boundary has high spatial frequency: grey values change abruptly.

Any image can be split into a low-frequency part (the broad pattern) and a high-frequency part (edges, lines, texture and noise). Spatial filters let us keep one part and suppress the other.

1.2 Low-pass and high-pass filters

A **low-pass filter (LPF)** keeps the low frequencies and removes the high ones. In practice it replaces each pixel by an average of its neighbours, so the image becomes smoother. Noise falls, but so does the sharpness of real edges.

A **high-pass filter (HPF)** does the reverse: it removes the broad pattern and keeps the abrupt changes. The classic remote sensing form is the original image minus a low-passed copy of it (Lillesand et al., 2015). Uniform areas become zero (mid-grey) and only edges remain. Adding a fraction of this edge layer back to the original gives a **high-boost** or edge-enhanced image, which looks sharper but keeps its colours.

Figure 1.1 shows both effects on real data. Along one line across the ridge, the low-pass curves (blue) are flatter than the original (black), while the high-boost curves (pink) are steeper and overshoot at every edge.

Both kinds of filter can be computed in two ways. In the *spatial domain* a small grid of

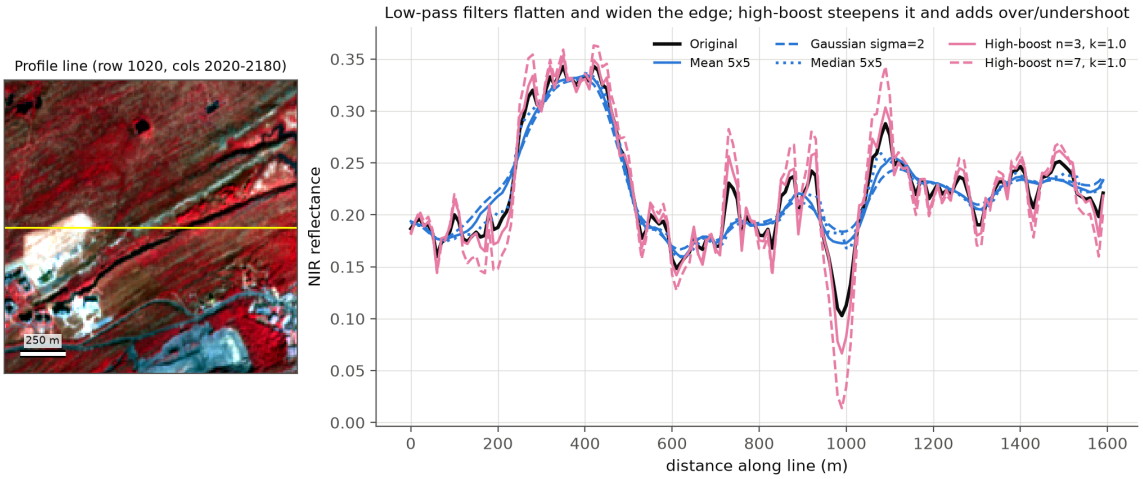


Figure 1.1: Near-infrared (NIR) reflectance along a 1.6 km line across a bright cleared patch on the ridge (yellow line, left). Low-pass filters flatten and widen each edge; high-boost filters steepen it and add overshoot.

weights (a kernel) slides over the image. In the *frequency domain* the image is transformed with the fast Fourier transform (FFT), the unwanted frequencies are multiplied by zero with a mask, and the result is transformed back (Gonzalez and Woods, 2018). Both routes appear in the course material, so we implemented both.

1.3 Aim

The project brief asked us to implement low-pass and high-pass filtering on remote sensing data and decide which technique is most suitable. Since smoothing and edge enhancement are meant for different jobs, we split the question by application:

1. Which filter removes noise best?
2. Which filter helps an automatic land-cover classification most?
3. Which filter makes edges, boundaries and geological lineaments easiest to see?
4. How much does each filter change the reflectance values that later analysis relies on?
5. Is the spatial or the frequency route faster?

Each question became one experiment (E1 to E5), and the answers were combined in a decision matrix (Chapter 5).

1.4 How this report is organised

Chapter 2 describes the study area and the satellite data. Chapter 3 explains the filters and the five experiments. Chapter 4 shows what the filters do and what each experiment found; every results section ends with a shaded box that states the main finding in one or two sentences. Chapter 5 brings the findings together in a decision matrix and a recommendation, and Chapter 6 sums up. Appendix A explains each evaluation measure in plain words, Appendix B shows where each step lives in the code, and Appendix C lists the commands that rebuild every result.

Chapter 2

Study Area and Data

2.1 Study area

The study area is a 25 km × 23 km box over the Aravalli ridge between Gurugram and Faridabad in Haryana (77.05–77.30 °E, 28.30–28.50 °N; Figure 2.1). We chose it because it holds very different textures in one scene:

- folded quartzite ridges with long, parallel linear features (high frequency, useful for testing HPF);
- the dense street grid of Gurugram (sharp man-made edges);
- small crop parcels in the south-east (smooth surfaces, useful for testing LPF);
- old quarries and a few ponds.

Three 2 km windows (ridge, urban, cropland; yellow boxes in Figure 2.1) are used for the zoomed comparisons in Chapter 4.

2.2 Satellite data and scene selection

We used Sentinel-2 Level-2A surface reflectance (Drusch et al., 2012; European Space Agency), downloaded from Google Earth Engine. The four 10 m bands were used: blue (B2), green (B3), red (B4) and near-infrared (B8).

Winter is usually the clearest season over Delhi NCR, so the program searched 1 November 2024 to 28 February 2025 for scenes with less than 5% cloud. 11 scenes qualified. Scene-wide cloud cover turned out to be a poor guide: inside our box the share of cloudy, hazy or missing pixels ranged from 0.08% to 97.2%, because winter smog does not count as cloud in the scene-wide figure. The program therefore scored every candidate inside the study area only, using the Sentinel-2 scene classification layer, the s2cloudless cloud

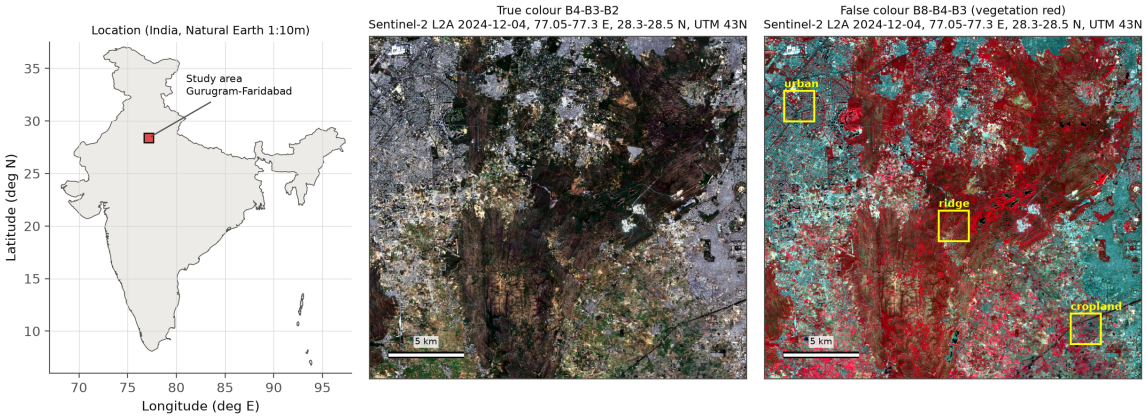


Figure 2.1: Location of the study area and the selected Sentinel-2 scene in true colour (red, green, blue) and false colour (NIR, red, green; vegetation appears red). Yellow boxes mark the ridge, urban and cropland windows.

probability and the aerosol optical thickness (AOT), and picked the cleanest, least hazy scene.

Table 2.1: Scene and data summary.

Item	Value
Product	`S2B_MSIL2A_20241204T053119_N0511_R105_T43RGM_20241204T074212`
Acquisition	2024-12-04, 05:41 UTC; tile 43RGM; processing baseline 05.11
Flagged pixels in the study area	0.08% (cloud, cirrus, shadow, no data or haze); mean AOT 0.164
Bands	B2, B3, B4, B8 at 10 m
Grid	2491 $\times$ 2261 pixels, UTM zone 43N (EPSG:32643), aligned to the native pixels
Reflectance	DN / 10000, clipped to [0, 1]
Land-cover reference	ESA WorldCover 2021 v200 (Zanaga et al., 2022), regridded by nearest neighbour
Demo subset	512 $\times$ 512 pixel crop with the most varied land cover

2.3 Land-cover reference

ESA WorldCover 2021 (Zanaga et al., 2022) gives a land-cover class for every 10 m pixel. We merged its classes into six groups: Trees, Shrub/Grass, Cropland, Built-up, Bare/Mining and Water. Water covered only 0.19% of the area and was dropped, which left five classes: Trees (19.9%), Shrub/Grass (28.7%), Cropland (24.6%), Built-up (25.3%) and Bare/Mining

(1.2%). WorldCover is itself produced by a classifier, so it is a reference map, not ground truth.

Chapter 3

Methods

3.1 Workflow

Figure 3.1 shows the processing chain. Every step is a Python function in the package `rsfilter`; one command (`python scripts/run_all.py`) repeats the whole analysis, and the random seed is fixed at 42.

All filters were written in NumPy. Unit tests compare them with the reference library `scipy.ndimage`; the largest difference is below 10^{-5}. Each filter works band by band on float32 reflectance. At the image border the image is mirrored, so no artificial black frame enters the result.

3.2 Spatial-domain filters

A spatial filter computes each output pixel $g(x,y)$ as a weighted sum of the input pixels f around it, with the weights held in a kernel w of size $(2a+1)\times(2b+1)$:

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\, f(x-s, y-t). \tag{3.1}$$

This is convolution; it turns the kernel by 180°, which makes no difference for the symmetric kernels used here. Instead of looping over pixels, our code loops over the few kernel cells: for each cell it shifts the whole padded image by (s,t), multiplies it by $w(s,t)$ and adds it to the output. This uses little memory and is easy to check.

Low-pass kernels. The *mean* filter uses equal weights, for example

$$w_{\text{mean},3} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \tag{3.2}$$

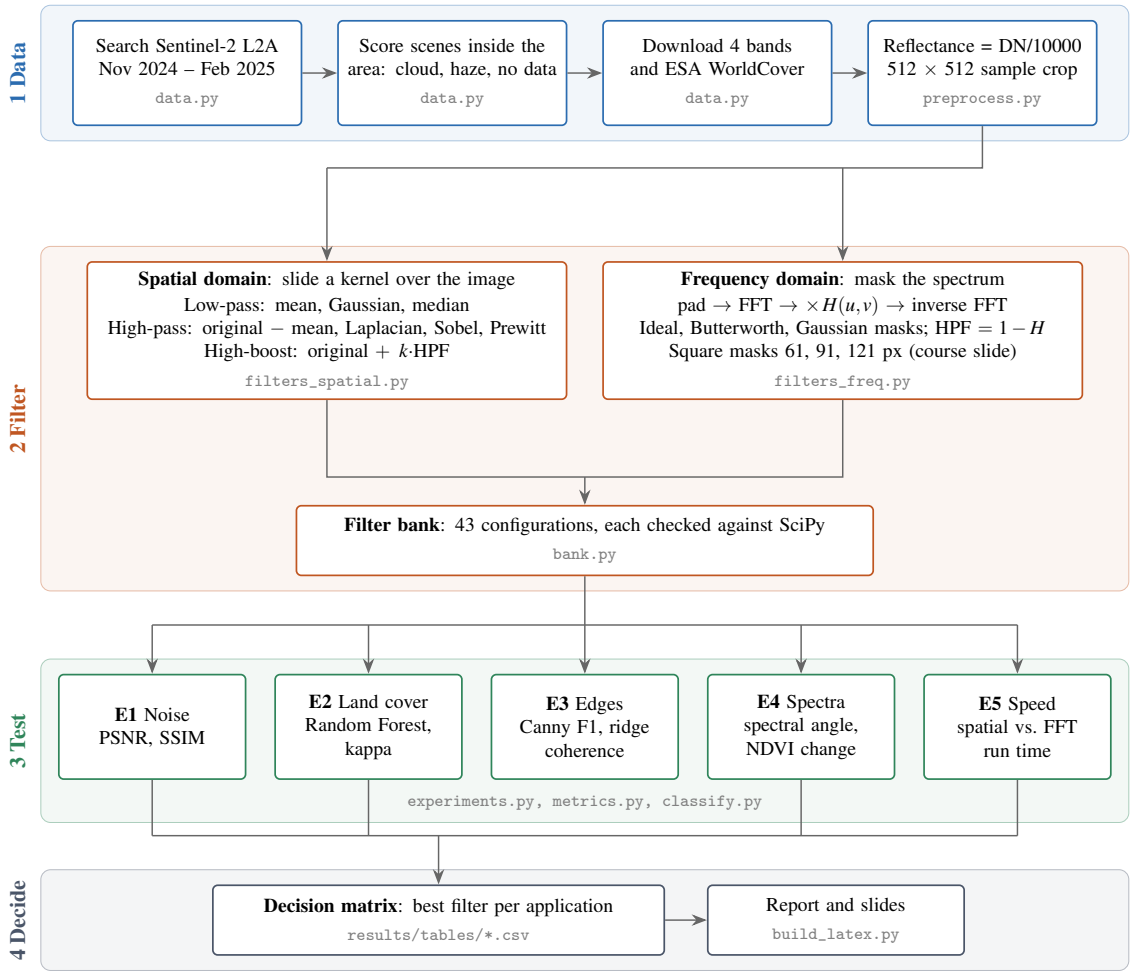


Figure 3.1: Processing flowchart. The work runs in four stages, top to bottom; the grey text in each box names the Python module that does the step (Appendix B).

The *Gaussian* filter gives nearby pixels more weight, $w(s,t) \propto \exp\big(-(s^2+t^2)/2\sigma^2\big)$, with a kernel side of $2\lceil 3\sigma \rceil + 1$ and weights that sum to 1. The *median* filter is not a weighted sum: it replaces each pixel by the median of its neighbourhood, which ignores single extreme values.

High-pass kernels. The remote sensing HPF subtracts a mean filter from the image (Lillesand et al., 2015):

$$\mathrm{HPF}_n = f - \mathrm{mean}_{n\times n}(f). \tag{3.3}$$

The Laplacian responds to a change in slope in any direction, and the Sobel and Prewitt operators measure the gradient in x and y; we use the gradient magnitude $\sqrt{G_x^2 + G_y^2}$:

$$w_{\mathrm{Lap},4} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \qquad w_{\mathrm{Sobel},x} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad w_{\mathrm{Prewitt},x} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}. \tag{3.4}$$

The 8-neighbour Laplacian also uses the diagonals; G_y uses the transposed kernel.

High-boost. An analyst rarely looks at the bare edge layer. High-boost adds it back to the image with a weight k:

$$\text{enhanced} = f + k \cdot \text{HPF}_n. \tag{3.5}$$

3.3 Frequency-domain filters

In the frequency domain, filtering is a multiplication (Gonzalez and Woods, 2018):

$$g = \mathscr{F}^{-1}\big\{H(u,v) \cdot \mathscr{F}\{f\}\big\}, \tag{3.6}$$

where $\mathscr{F}$ is the 2-D FFT and H is a mask that is 1 for frequencies we keep and 0 (or close to 0) for those we remove. Our pipeline per band is: mirror-pad by 64 pixels, FFT, shift the zero frequency to the centre, multiply by H, shift back, inverse FFT, keep the real part, and crop the padding. The padding matters: the FFT treats the image as if it repeated forever, so without padding the left border would leak into the right border.

Let D be the distance from the centre of the spectrum, scaled so that $D = 1$ at the Nyquist frequency (the highest frequency a 10 m grid can hold), and D_0 the cutoff. We used three low-pass masks:

$$\text{Ideal:} \quad H = 1 \text{ if } D \le D_0, \text{ else } 0 \tag{3.7}$$

$$\text{Butterworth (order 2):} \quad H = 1/\big(1 + (D/D_0)^4\big) \tag{3.8}$$

$$\text{Gaussian:} \quad H = \exp\big(-D^2/2D_0^2\big) \tag{3.9}$$

with $D_0 = 0.1$, 0.2 and 0.3. Each high-pass mask is $1 - H$. A Gaussian mask with cutoff D_0 is the same filter as a spatial Gaussian with $\sigma = 1/(\pi D_0)$ pixels, which gave us a direct check between the two domains. To reproduce the example in the course slides, we also applied square ideal low-pass masks of 61, 91 and 121 pixels to the 512×512 crop.

3.4 Experiments

Table 3.2 lists the five experiments. Each writes one table to `results/tables/`, and every number in this report is read from those tables by a script.

Two choices in E2 and E3 need a short explanation.

Blocked train/test split (E2). Neighbouring pixels are very similar. If training and test pixels are drawn at random, almost every test pixel has a near-copy in the training set, and accuracy looks better than it is. We therefore cut the area into 128×128 pixel blocks and

Table 3.1: Filter bank: the 43 configurations compared.

Family	Filter	Parameters
Low-pass, spatial	Mean	$n = 3, 5, 7, 9$
	Gaussian	$\sigma = 1, 2, 3$ pixels
	Median	$n = 3, 5$
Low-pass, frequency	Ideal, Butterworth ($n = 2$), Gaussian	$D_0 = 0.1, 0.2, 0.3$ of Nyquist
High-pass, spatial	Original $-$ mean	$n = 3, 5, 7$
	Laplacian	4- and 8-neighbour
	Sobel, Prewitt	gradient magnitude
High-pass, frequency	Ideal, Butterworth, Gaussian	$1 - H$, $D_0 = 0.1, 0.2, 0.3$
High-boost	$f + k \cdot \mathrm{HPF}_n$	$n = 3, 5, 7$; $k = 0.5, 1.0, 1.5$

Table 3.2: The five experiments.

	Question and set-up	Measure
E1	Noise removal. Add Gaussian noise ($\sigma = 0.02$) or 2% salt-and-pepper noise to the clean 512×512 crop, then apply each low-pass filter.	PSNR and SSIM (Wang et al., 2004) against the clean crop (higher is better).
E2	Classification. Random Forest (Breiman, 2001), 200 trees, 2000 training pixels per class, on the four bands plus NDVI computed after filtering. Inputs: raw bands, each LPF, each high-boost image, and raw bands with HPF layers added.	Overall accuracy, Cohen's κ, macro-F1; salt-and-pepper index = share of pixels whose class differs from most of their 3×3 neighbours.
E3	Edges and lineaments on the NIR band.	Average gradient; F1 of Canny edges (Canny, 1986) against WorldCover class boundaries (2-pixel tolerance); orientation coherence on the ridge (Bigün and Granlund, 1987).
E4	Spectral change caused by the filter.	Correlation, RMSE, spectral angle (SAM), change in NDVI.
E5	Speed on one full band (2261×2491 pixels).	Median of 5 runs, spatial vs. frequency route.

gave 70% of the blocks to training and 30% to testing (10 000 training and 1 700 310 test pixels before sampling).

Orientation coherence (E3). The study area has no published lineament map, so we needed a measure of how strongly linear an image is. Coherence, computed from the structure tensor in a 2×2 km ridge window, is 1 when all local gradients point the same way (parallel ridges) and 0 when they point in random directions (speckle).

Chapter 4

Results

4.1 What the filters do to the image

Figure 4.1 shows low-pass results on the ridge and urban windows. All low-pass filters blur the street grid and field edges; the broad shape of the ridge survives. The median keeps straight edges a little sharper than a mean or Gaussian of the same size, because it picks an existing value rather than averaging across the edge. The ideal frequency mask leaves a faint ripple texture; the Gaussian frequency mask looks like the spatial Gaussian, as expected.

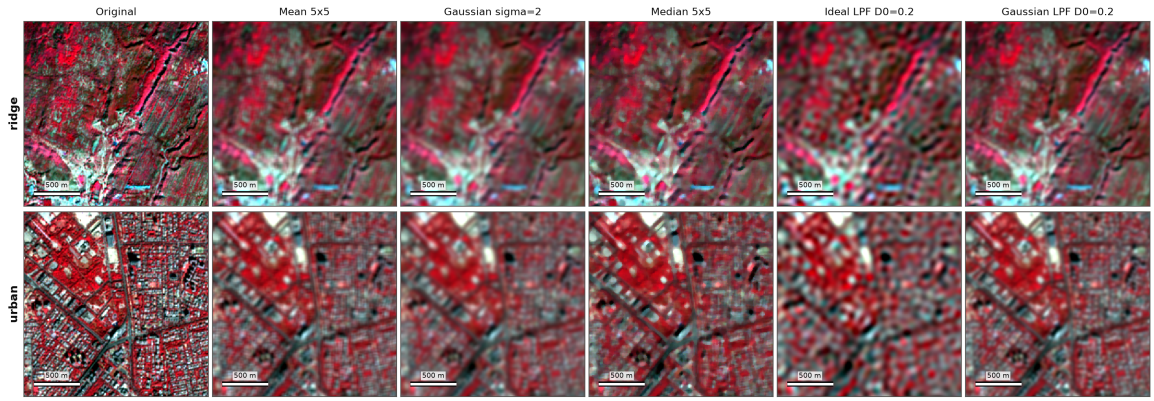


Figure 4.1: Low-pass results on 2 km windows over the ridge (top) and Gurugram (bottom), false colour, same colour stretch in every panel.

High-pass outputs (Figure 4.2) are edge maps: flat ground turns mid-grey and only changes remain. The 7×7 HPF and the low-cutoff frequency HPF bring out the ridge crests. The Laplacian reacts most to fine speckle. Sobel shows edge strength without regard to direction. High-boost images (Figure 4.3) keep the colours and sharpen boundaries; at $k = 1.5$ bright and dark halos appear along roads and ridge crests.

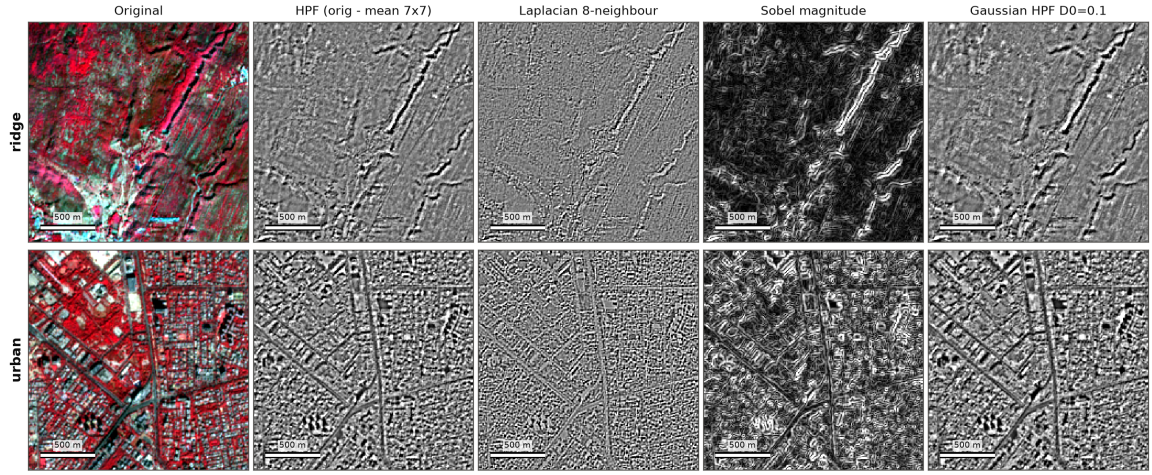


Figure 4.2: High-pass results on the NIR band for the same two windows.

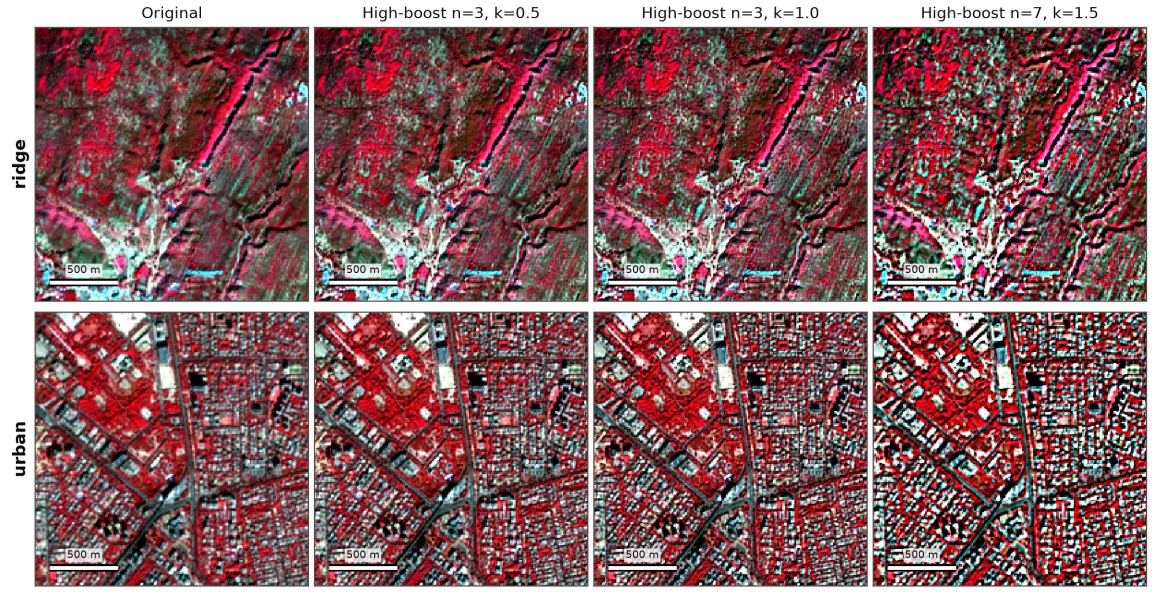


Figure 4.3: High-boost (original $+k\cdot$HPF) for increasing k, false colour.

4.2 Frequency domain and ringing

Figure 4.4 reproduces the course slide. The smaller the square mask, the fewer frequencies pass and the blurrier the result; all three results show a ripple texture.

These ripples are *ringing*. A mask with a sharp edge in frequency produces oscillations beside every edge in the image (the Gibbs effect; Gonzalez and Woods, 2018). On real data the ripples are hidden by texture, so we passed a clean step edge through the same code (Figure 4.5). At $D_0 = 0.1$ the ideal mask overshot by 9.0% of the step height, Butterworth by 3.4% and Gaussian by 0.0%. The ideal value agrees with the roughly 9% predicted by theory.

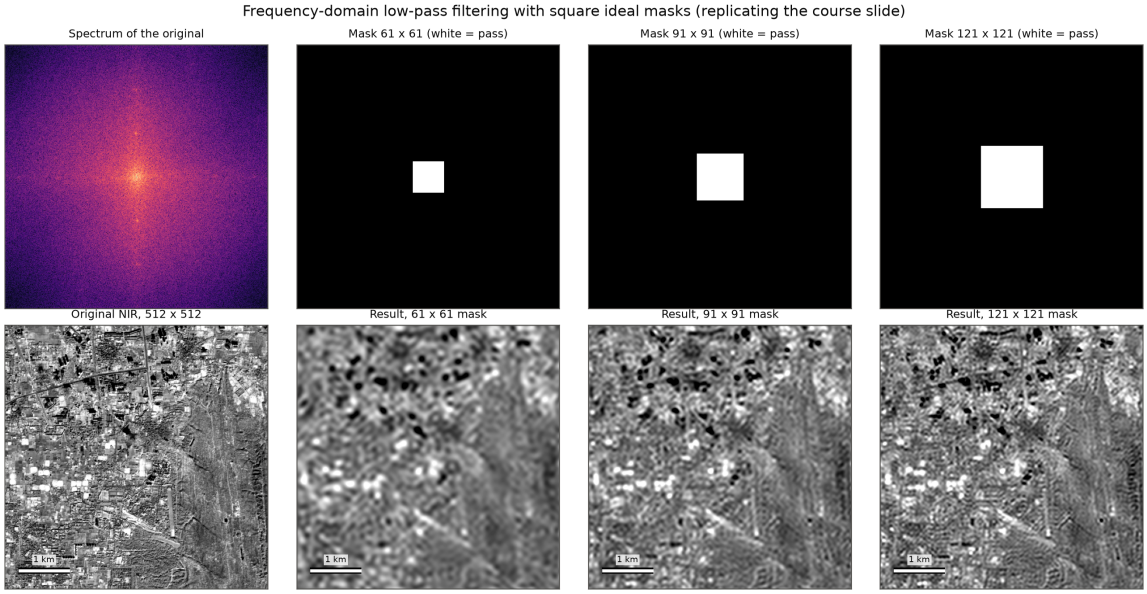


Figure 4.4: Reproduction of the course slide: square ideal low-pass masks of 61, 91 and 121 pixels (top) and the filtered NIR crop (bottom).

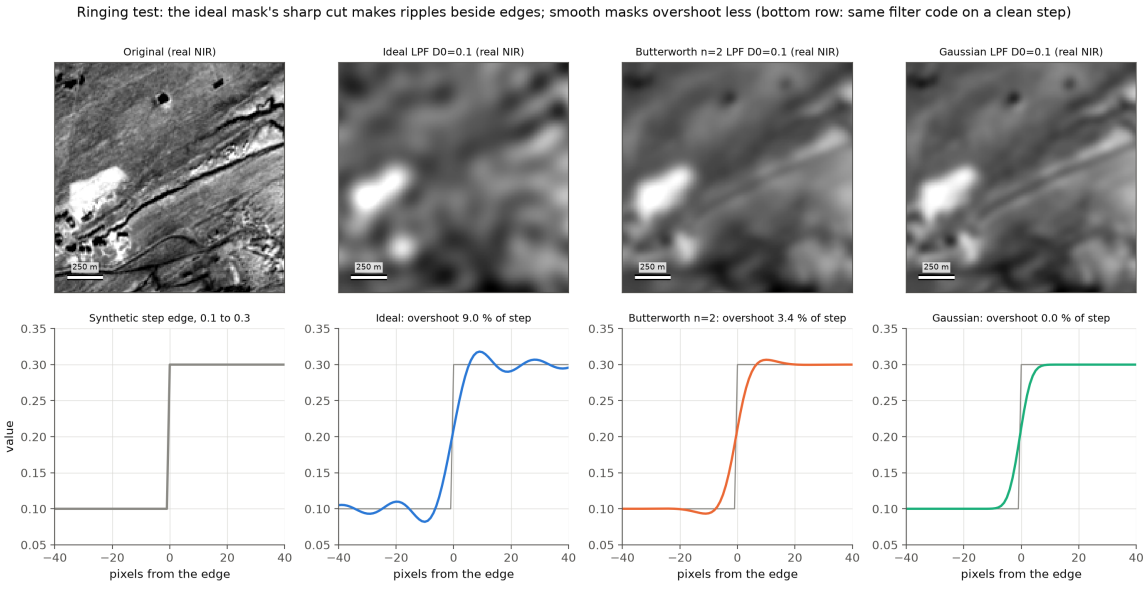


Figure 4.5: Ringing. Top: NIR window after each low-pass mask ($D_0 = 0.1$). Bottom: the same masks applied to a clean step edge.

A sharp cut in frequency always means ripples in space. Use a Gaussian or Butterworth mask, never the ideal one.

4.3 E1: Noise removal

Table 4.1 gives the results for a representative set of filters. With random (Gaussian) noise the noisy image had a PSNR of 33.98 dB. Only the lightest filters improved on it: Gaussian $\sigma = 1$ reached 35.56 dB, and Mean 3×3 35.52 dB. 13 of the 18 low-pass filters made the image worse than doing nothing, because they removed more real detail than noise.

Salt-and-pepper noise behaved very differently. The 3×3 median raised PSNR from 21.01 dB to 36.49 dB, 5.42 dB ahead of the best linear filter (Gaussian LPF $D_0 = 0.2$). A mean or Gaussian spreads each spike over its neighbours; the median simply discards it.

Table 4.1: E1: noise removal on the 512×512 crop. Best value per column in bold. Full results: `e1_noise.csv`.

	Gaussian noise, $\sigma = 0.02$		Salt-and-pepper, 2%	
Filter	PSNR (dB)	SSIM	PSNR (dB)	SSIM
Noisy input (no filter)	33.98	0.825	21.01	0.603
Mean 3×3	35.52	0.882	29.29	0.714
Mean 5×5	32.60	0.775	30.39	0.695
Gaussian $\sigma = 1$	**35.56**	**0.885**	30.27	0.748
Gaussian $\sigma = 2$	32.26	0.765	30.91	0.728
Median 3×3	35.19	0.872	**36.49**	**0.910**
Median 5×5	32.65	0.779	32.89	0.794
Ideal LPF $D_0 = 0.3$	33.05	0.799	29.65	0.676
Butterworth LPF $D_0 = 0.3$	34.33	0.848	30.71	0.746
Gaussian LPF $D_0 = 0.3$	35.22	0.876	30.48	0.749

Median 3×3 for impulse noise; a light Gaussian ($\sigma = 1$) or 3×3 mean for random noise. Larger kernels do more harm than good.

4.4 E2: Land-cover classification

On raw bands the Random Forest reached an overall accuracy of 0.536 and $\kappa = 0.403$ on the held-out blocks (Table 4.2). Every low-pass input improved on that. The best input

was the Gaussian LPF $D_0 = 0.2$ (accuracy 0.615, $\kappa = 0.502$), almost tied with the spatial Gaussian $\sigma = 2$ ($\kappa = 0.500$). This tie is expected: the two are nearly the same filter, since $\sigma = 1/(\pi \times 0.2) \approx 1.6$ pixels. Smoothing also cleaned up the map: the salt-and-pepper index fell from 0.263 to 0.118 (Figure 4.6).

High-boost inputs had the opposite effect. The best one (High-boost $n = 3$, $k = 0.5$) reached only $\kappa = 0.360$ and the worst 0.275. Adding HPF layers next to the raw bands helped ($\kappa = 0.481$), but less than smoothing.

The reason is that at 10 m a single class contains many different surfaces: roofs of different materials, shadows, mixed pixels at field edges. A pixel-by-pixel classifier is confused by this variation. Smoothing averages it out before the classifier sees it; sharpening makes it larger.

Table 4.2: E2: classification on held-out blocks. Best and worst input of each family; best value per column in bold (lowest for the salt-and-pepper index). Full results: `e2_classification.csv`.

Input	Family	Accuracy	κ	Macro-F1	Salt-and-pepper
No filter	none	0.536	0.403	0.473	0.263
Gaussian $\sigma = 2$	LPF, spatial	0.613	0.500	0.539	**0.113**
Gaussian LPF $D_0 = 0.2$	LPF, frequency	**0.615**	**0.502**	**0.540**	0.118
High-boost $n = 3$, $k = 0.5$	high-boost	0.500	0.360	0.444	0.316
High-boost $n = 7$, $k = 1.5$	high-boost	0.431	0.275	0.383	0.378
Raw + HPF 7×7 layers	raw + HPF stack	0.598	0.481	0.528	0.178

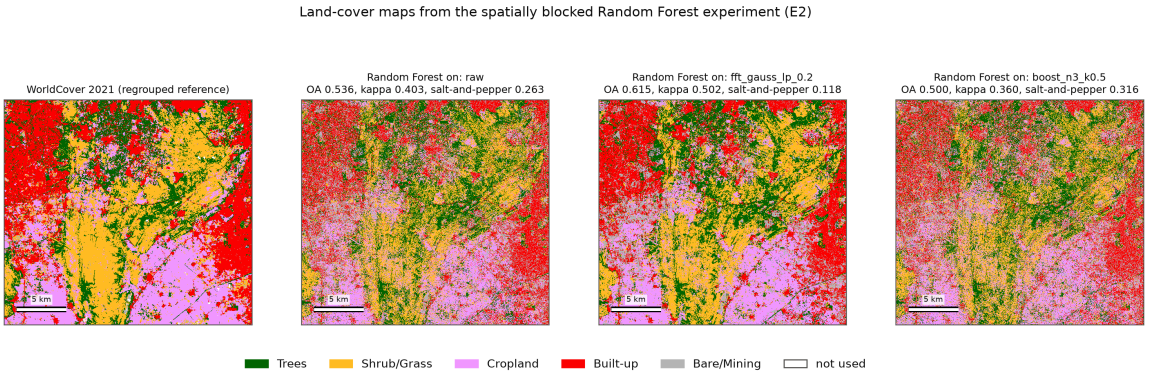


Figure 4.6: WorldCover reference and Random Forest maps from raw bands, the best low-pass input and the best high-boost input.

For classification, smooth first. A light Gaussian raised κ by 0.099 and halved map speckle; high-boost lowered κ.

4.5 E3 and E4: Edges, lineaments and spectral change

Table 4.3 puts the edge measures (E3) next to the cost in spectral fidelity (E4) for a set of filters.

Boundaries. Canny edges from the original NIR band matched WorldCover class boundaries with F1 = 0.675. High-boost raised this to 0.742 (High-boost $n = 5$, $k = 1.5$); low-pass filters lowered it (0.247 for the 5×5 mean), because fewer edges survive smoothing. For built-up boundaries alone the gain was negligible (0.545 for High-boost $n = 3$, $k = 0.5$ against 0.543).

Lineaments. On the ridge, the original image had a coherence of 0.510. Among high-pass outputs only the gradient operators clearly raised it (Prewitt 0.536, Sobel 0.535). The Laplacian scored lowest (0.426): it responds to the finest texture, which on the ridge is rock and scrub speckle with no common direction. Low-pass outputs scored even higher (0.754 for Gaussian $\sigma = 3$), but only because they erase that speckle along with fine detail; they do not show the lineaments more clearly.

Spectral change. Light low-pass filters changed the spectra little: a spectral angle of 2.22° for the 3×3 median and 2.30° for Gaussian $\sigma = 1$. Mild high-boost ($n = 3$, $k = 0.5$) changed them even less (1.20°). Pure HPF outputs are not reflectance any more: their mean is near zero, the spectral angle to the original is about 89°, and an NDVI computed from them has no meaning.

High-pass filtering helps wherever the task is about *where things change*: boundaries, ridges, visual sharpness. Keep HPF outputs as separate layers; they cannot replace the bands.

4.6 E5: Run time

On one full 2261×2491 band the spatial mean filter took 0.10 s with a 3×3 kernel and 8.88 s with 31×31; the spatial Gaussian took 20.96 s at $\sigma = 8$ (49×49 kernel). Every frequency-domain filter took 1.36–1.48 s, whatever the mask or cutoff (Figure 4.7). Spatial cost grows with the square of the kernel side; FFT cost does not depend on it. The two cross at a kernel side of about 13 pixels.

Table 4.3: E3 and E4 for selected filters on the NIR band. Edge F1 values are only defined for outputs that are still images. SAM: mean spectral angle to the original; ΔNDVI: mean absolute change in NDVI. Full results: `e3_edges.csv`, `e4_fidelity.csv`.

Filter	Mean gradient	Boundary F1	Built-up F1	Ridge coherence	SAM (°)	ΔNDVI
No filter	0.0245	0.675	0.543	0.510	–	–
Gaussian $\sigma = 1$	0.0103	0.479	0.430	0.607	2.30	0.037
Median 3×3	0.0129	0.574	0.491	0.595	2.22	0.033
Mean 5×5	0.0070	0.247	0.244	0.643	3.29	0.054
Gaussian LPF $D_0 = 0.2$	0.0063	0.223	0.222	**0.653**	3.16	0.052
HPF 7×7	0.0241	–	–	0.502	88.76	0.569
Laplacian (4-neighbour)	0.0715	–	–	0.426	91.62	0.543
Sobel	**0.0926**	–	–	0.535	18.90	0.331
Prewitt	0.0656	–	–	0.536	19.00	0.333
Gaussian HPF $D_0 = 0.1$	0.0239	–	–	0.508	89.18	0.588
High-boost $n = 3$, $k = 0.5$	0.0330	0.706	**0.545**	0.486	1.20	0.019
High-boost $n = 3$, $k = 1.0$	0.0417	0.725	0.537	0.472	2.52	0.039
High-boost $n = 5$, $k = 1.5$	0.0589	**0.742**	0.477	0.487	7.25	0.103

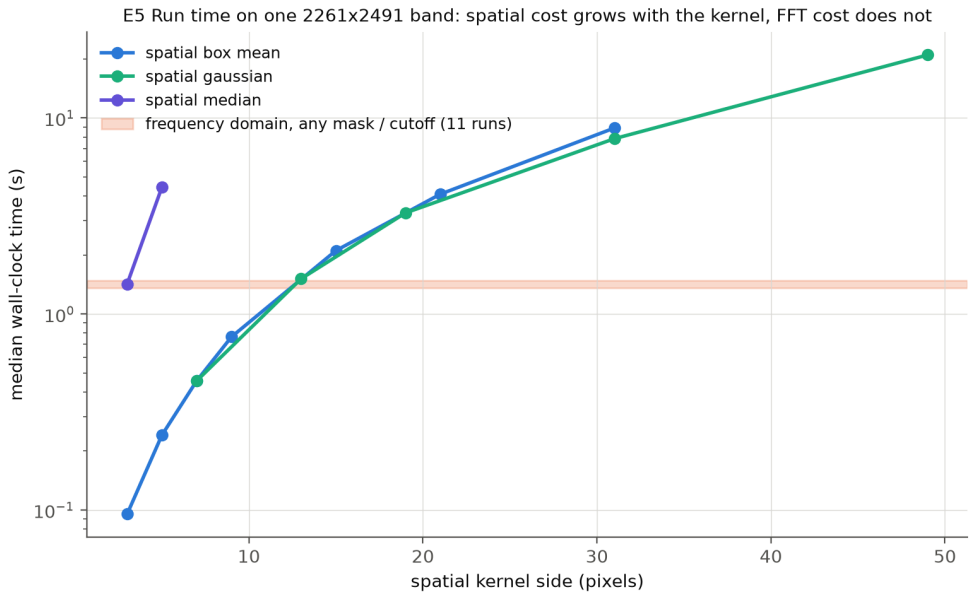


Figure 4.7: Run time on one full band against kernel size. The shaded band is the range of all frequency-domain runs.

Chapter 5

Discussion

5.1 Decision matrix

Table 5.1 collects the answers. For each application one measure decides the winner, the "No filter" column shows what the unfiltered image scores, and the last column gives the cost of the winner as spectral angle. The table is generated by the analysis code (`decision_matrix.csv`).

Table 5.1: Decision matrix: best filter for each application.

Application	Best filter	Measure	Best	No filter	Runner-up	SAM (°)
Noise removal, random noise	Gaussian $\sigma = 1$	PSNR (dB)	35.56	33.98	Mean 3×3 (35.52)	2.30
Noise removal, impulse noise	Median 3×3	PSNR (dB)	36.49	21.01	Median 5×5 (32.89)	2.22
Land-cover classification	Gaussian LPF $D_0 = 0.2$	Cohen's κ	0.502	0.403	Gaussian $\sigma = 2$ (0.500)	3.16
Lineament mapping (ridges)	Prewitt	ridge coherence	0.536	0.510	Sobel (0.535)	19.00
Urban boundary delineation	High-boost $n = 3, k = 0.5$	built-up edge F1	0.545	0.543	High-boost $n = 3$, $k = 1.0$ (0.537)	1.20
Visual interpretation	High-boost $n = 3, k = 1.0$	NIR mean gradient	0.0417	0.0245	High-boost $n = 7$, $k = 0.5$ (0.0364)	2.52

5.2 When to use which filter

Low-pass filtering suits tasks that need clean, stable values per pixel: classification, vegetation indices, change detection. It removed noise and raised classification κ by 0.099.

The cost is lost boundary detail, and strong smoothing removed real detail faster than noise (E1). Light filters are therefore the right choice.

High-pass filtering suits tasks about where things change. High-boost with k between 0.5 and 1 gave the best visual sharpening (High-boost $n = 3$, $k = 1.0$: 1.70 times the original gradient at a spectral angle of 2.52°) and the best agreement between detected edges and mapped boundaries. For the Aravalli ridges, the Sobel and Prewitt gradient magnitudes brought out the most coherent linear structure. HPF outputs should be stored beside the bands, not instead of them.

Spatial or frequency domain. With a smooth mask, both routes give the same result (the frequency and spatial Gaussians tie in E2). The frequency route is worth using for large kernels, or when many cutoffs are tried on the same image, because its cost does not grow with the kernel. For the small kernels that work best in practice, convolution is faster and simpler.

5.3 Recommendation

Which filter for which task

- **Quantitative analysis** (classification, indices, change detection): light low-pass, i.e. Gaussian with $\sigma \approx 1.5$–2 pixels, or the frequency Gaussian with $D_0 \approx 0.2$.
- **Impulse noise** (dropped or saturated pixels): 3×3 median.
- **Visual interpretation**: high-boost with $k = 0.5$–1, added on top of the bands.
- **Lineament and structural mapping**: Sobel or Prewitt gradient magnitude as a separate layer.
- **Avoid** the ideal frequency mask for both low-pass and high-pass work because of ringing.

5.4 Limitations

- WorldCover is a model output (ESA reports about 77% overall accuracy for v200), so E2 and the boundary scores measure agreement with a reference map, not with the ground.
- We used one winter scene. Haze, crop stage and soil moisture change through the year, and a second date or a second area would test how general the ranking is.

- E2 used one random block split. Differences in κ of about 0.01 between the leading filters should not be read as a ranking.
- Ridge coherence in one 2×2 km window stands in for lineament quality because no reference lineament map exists for the area.
- Bare/Mining covered only 1.2% of the area and was classified poorly by every input (F1 at most 0.14).

Chapter 6

Summary

We implemented low-pass and high-pass filters in both the spatial and the frequency domain and tested 43 configurations on a Sentinel-2 scene of the Aravalli ridge. The question "which is most suitable" has no single answer, because the two families do opposite jobs. Smoothing helps when pixel values feed a calculation: a light Gaussian gave the most accurate and cleanest land-cover map, and a 3×3 median removed impulse noise. Edge enhancement helps when edges are the point: high-boost gave the sharpest images and best boundary match, and gradient operators gave the clearest ridge lineaments. Among frequency-domain masks, Gaussian and Butterworth are safe while the ideal mask rings. The spatial route is faster for small kernels and the FFT route for kernels of about 13 pixels and more.

Appendix A

Evaluation Measures in Plain Words

Table A.1: What each evaluation measure tells us.

Measure	What it tells us	Better when
PSNR (dB)	How close the filtered image is to the clean image. It is based on the mean squared error; 3 dB more means about half the error.	higher
SSIM	How similar the local patterns (brightness, contrast, structure) are to the clean image, from 0 to 1.	closer to 1
Overall accuracy	Share of test pixels whose class matches WorldCover.	higher
Cohen's κ	Accuracy after removing the agreement expected by pure chance: 0 is chance level, 1 is perfect.	higher
Macro-F1	F1 score averaged over the classes, so a small class counts as much as a large one.	higher
Salt-and-pepper index	Share of map pixels whose class differs from most of their 3×3 neighbours. Measures how speckled a map looks.	lower
Boundary F1	Balance of two shares: detected edges that lie on a mapped class boundary (precision), and mapped boundaries that were detected (recall). A 2-pixel offset is allowed.	higher
Average gradient	Mean change in brightness between neighbouring pixels. Measures sharpness.	higher for sharpening
Orientation coherence	Whether local edges share one direction (1, parallel ridges) or point every way (0, speckle).	higher for lineaments
Spectral angle (SAM)	Angle between a pixel's four-band spectrum before and after filtering. 0° means the shape of the spectrum is unchanged.	lower
NDVI change	Mean absolute change in NDVI caused by the filter.	lower

Measure	What it tells us	Better when
Overshoot	How far a filtered step edge rises past its true level, as a percentage of the step height.	lower

Appendix B

Where Each Step Lives in the Code

All code is in the Python package `src/rsfilter/`. Every module and public function has a docstring that explains what it does and why, with formulas where they apply. Table B.1 maps the steps of Figure 3.1 to the code and to the files each step writes.

Table B.1: Map from processing step to code and output.

Step	Module and main functions	Output
Find and rank scenes, download	`data.py`: `fetch_scene`, `rank_candidates`	`data/raw/`, `scene_candidates.csv`
Reflectance, WorldCover regrid, sample crop	`preprocess.py`: `build_processed`, `to_reflectance`, `warp_to_grid`, `make_sample`	`data/processed/`, `data/sample/`
Spatial filters	`filters_spatial.py`: `convolve2d`, `mean_filter`, `gaussian_filter`, `median_filter`, `high_pass`, `laplacian`, `sobel`, `prewitt`, `high_boost`	filtered arrays
Frequency filters	`filters_freq.py`: `freq_filter`, `make_mask`, `square_lowpass`, `step_response`	filtered arrays
List of all filters	`bank.py`: `filter_bank`	named configurations
Classification	`classify.py`: `regroup`, `block_split`, `run_classification`	`results/maps/`
Measures	`metrics.py`: `psnr`, `ssim`, `boundary_prf`, `coherence`, `spectral_angle`, …	–
Experiments E1–E5	`experiments.py`: `e1_noise`, `e2_classification`, `e3_e4`, `e5_runtime`, `ringing_table`, `decision_matrix`	`results/tables/*.csv`
Figures	`viz.py`	`results/figures/`
Command line	`cli.py`: `fetch`, `filter`, `evaluate`, `figures`, `demo`	–

Step	Module and main functions	Output
Report and slides	`scripts/build_latex.py`	`report/`, `presentation/`
Tests	`tests/`: our filters against `scipy.ndimage`, masks, measures	–

Appendix C

Reproducing the Results

The code, the 512×512 sample data and all result tables are in the project repository. The main commands are:

Command	What it does
`pip install -r requirements.txt`	install the pinned Python packages
`python -m rsfilter fetch`	download the scene and WorldCover (Earth Engine, or open STAC servers as fallback)
`python scripts/run_all.py`	run E1–E5 and the decision matrix on the full area
`python scripts/make_figures.py`	redraw all figures
`python -m rsfilter demo`	run the whole pipeline offline on the sample crop in about a minute
`python -m rsfilter filter ...`	apply one filter to a GeoTIFF and keep its georeferencing
`pytest -q`	check every filter against `scipy.ndimage`
`python scripts/build_latex.py`	rebuild this report and the slides from the result tables

The algorithm with all formulas, the pseudo-code with array shapes and data types, and a module-by-module code reference are in `docs/algorithm.md`, `docs/pseudocode.md` and `docs/code_reference.md`.

Bibliography

Josef Bigün and Gösta H. Granlund. Optimal orientation detection of linear symmetry. In *Proceedings of the First International Conference on Computer Vision*, pages 433–438, London, 1987.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.

M. Drusch, U. Del Bello, S. Carlier, O. Colin, V. Fernandez, F. Gascon, B. Hoersch, C. Isola, P. Laberinti, P. Martimort, A. Meygret, F. Spoto, O. Sy, F. Marchese, and P. Bargellini. Sentinel-2: ESA's optical high-resolution mission for GMES operational services. *Remote Sensing of Environment*, 120:25–36, 2012.

European Space Agency. Sentinel-2 Level-2A Product Specification and Sen2Cor Algorithm Theoretical Basis Document. ESA Sentinel Online.

Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, New York, 4th edition, 2018.

Thomas M. Lillesand, Ralph W. Kiefer, and Jonathan W. Chipman. *Remote Sensing and Image Interpretation*. Wiley, Hoboken, NJ, 7th edition, 2015.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

Daniele Zanaga, Ruben Van De Kerchove, Dirk Daems, Wanda De Keersmaecker, Carsten Brockmann, Grit Kirches, Jan Wevers, Oliver Cartus, Maurizio Santoro, Steffen Fritz, et al. ESA WorldCover 10 m 2021 v200. Zenodo, 2022.